UNIVERSIDADE FEDERAL DE MINAS GERAIS

Gabriel Henrique Silva Maia

Automatização do envio de relatórios para e-mail com Python

Belo Horizonte, Minas Gerais, Brasil

21 de janeiro de 2025

UNIVERSIDADE FEDERAL DE MINAS GERAIS

Gabriel Henrique Silva Maia

Automatização do envio de relatórios para e-mail com Python

Trabalho final da disciplina Métodos e Ciências dos Dados ofertada pelo Departamento de Ciências Econômicas da UFMG.

Departamento de Ciências Econômicas

Métodos e Ciências dos Dados

Professores:
Leonardo Costa Ribeiro
Lucas Resende de Carvalho

Belo Horizonte, Minas Gerais, Brasil

21 de janeiro de 2025

Sumário

Sumário 2

1 INTRODUÇÃO 4

2 FUNDAMENTAÇÃO TEÓRICA 5
2.1 Automação de Processos 5
2.2 Banco de Dados Relacional e SQL 5
2.3 Manipulação de Dados e Automação com Python 6

3 ESTRUTURA DA BASE DE DADOS 8
3.1 Estrutura da Base de Dados 8

4 DESENVOLVIMENTO DA SOLUÇÃO 12
4.1 Desenvolvimento da Solução 12
4.1.1 Conexão com o Banco de Dados 12
4.1.2 Consultas SQL para Extração de Dados 12
4.1.3 Geração de Relatórios com Pandas e Exportação para Excel 13
4.1.4 Implementação do Envio Automatizado de E-mails 13
4.1.5 Considerações Finais 14

5 RESULTADOS E TESTES 19
5.1 Resultados e Testes 19
5.1.1 Exemplos de Relatórios Gerados 19
5.1.2 Validação dos Dados Extraídos 23
5.1.3 Testes do Envio Automático de E-mails 23
5.1.4 Benefícios da Automação para a Empresa 25

6 CONCLUSÃO 26
6.1 Conclusão 26
6.1.1 Resumo dos Resultados Alcançados 26
6.1.2 Impacto da Automação no Processo de Envio de Relatórios 26
6.1.3 Possíveis Melhorias e Extensões Futuras 27

7 REFERÊNCIAS 28

Lista de ilustrações

Figura 1 – Criação das Tabelas - 1 9
Figura 2 – Criação das Tabelas - 2 10
Figura 3 – Criação das Tabelas - 3 10
Figura 4 – Código Python - 1 14
Figura 5 – Código Python - 2 15
Figura 6 – Código Python - 3 16
Figura 7 – Código Python - 4 17
Figura 8 – Código Python - 5 18
Figura 9 – Código Python - 6 18
Figura 10 – Relatório turnover 20
Figura 11 – Relatório departamentos 21
Figura 12 – Relatório demográfico 22
Figura 13 – E-mail enviado 24

1 Introdução

No cenário atual, a automação de processos tornou-se essencial para aumentar a eficiência e reduzir erros operacionais em diversas áreas empresariais. No setor de Recursos Humanos, a geração e o envio de relatórios periódicos são tarefas fundamentais para a gestão de colaboradores, acompanhamento de indicadores e tomada de decisões estratégicas. No entanto, esses processos, quando realizados manualmente, podem ser demorados e suscetíveis a falhas.

Este trabalho aborda a implementação de uma solução automatizada para a geração e envio de relatórios utilizando a linguagem de programação Python. A partir de uma base de dados estruturada em SQL, foi desenvolvido um script que realiza consultas em tabelas relacionadas a contratos, cargos, histórico funcional e folha de pagamento, gerando relatórios analíticos sobre o quadro de funcionários. Esses relatórios são exportados para um arquivo Excel e, em seguida, enviados automaticamente por e-mail para os responsáveis pelo setor de RH.

A automação proposta utiliza bibliotecas como pandas para manipulação e exportação de dados, psycopg2 para conexão com o banco de dados PostgreSQL e smtplib para envio de e-mails. A implementação visa não apenas otimizar o tempo gasto na geração dos relatórios, mas também garantir maior precisão e confiabilidade das informações enviadas periodicamente.

Nos capítulos seguintes, serão detalhadas a estrutura da base de dados utilizada, a lógica empregada para extração e transformação dos dados, além do código desenvolvido para a automatização completa do processo.

2 Fundamentação Teórica

Pouco se discute sobre a importância da automação no setor de Recursos Humanos (RH) e os impactos positivos que essa transformação pode trazer. Com o avanço da tecnologia e a crescente necessidade de otimização de processos, a automação de relatórios no RH se torna uma ferramenta essencial para garantir eficiência, reduzir erros e permitir uma melhor tomada de decisão baseada em dados.

Este tema foi dividido em tópicos principais para uma melhor exploração da literatura. A Seção 2.1 aborda os conceitos de automação de processos e suas aplicações no contexto organizacional. Logo mais, a Seção 2.2 apresenta uma introdução ao Banco de Dados Relacional, com foco no uso de SQL e PostgreSQL. Por fim, a Seção 2.3 discute a manipulação de dados e o envio automatizado de relatórios por meio da linguagem Python e suas principais bibliotecas.

2.1 Automação de Processos

A automação de processos pode ser definida como a utilização de tecnologia para executar tarefas repetitivas e operacionais de forma automática, minimizando a necessidade de intervenção humana. No contexto do RH, a automação permite que atividades como geração de relatórios, análise de desempenho e controle de folha de pagamento sejam realizadas de maneira eficiente e com maior precisão.

A implementação da automação traz benefícios como a redução de erros humanos, otimização do tempo de trabalho e melhor organização dos dados. Segundo (**??**), a automação dos processos empresariais permite que as equipes de RH concentrem seus esforços em atividades estratégicas, como o desenvolvimento de talentos e a melhoria do ambiente organizacional.

2.2 Banco de Dados Relacional e SQL

Os bancos de dados relacionais são amplamente utilizados para armazenar e gerenciar informações estruturadas. Eles se baseiam no modelo relacional proposto por Edgar F. Codd na década de 1970 e utilizam a Linguagem de Consulta Estruturada (SQL) para manipulação de dados.

O PostgreSQL é um dos sistemas de gerenciamento de banco de dados relacionais mais utilizados devido à sua confiabilidade, robustez e capacidade de lidar com grandes volumes de dados. No contexto da automação de relatórios no RH, o PostgreSQL desem-

penha um papel fundamental ao armazenar e organizar dados sobre funcionários, cargos, contratos e outros elementos essenciais.

Com a utilização de consultas SQL, é possível extrair informações específicas de forma rápida e precisa. Por exemplo, a consulta abaixo retorna uma lista de funcionários ativos em determinado departamento:

```
SELECT nome, cargo, salario
FROM funcionarios
WHERE status = 'Ativo' AND departamento = 'RH';
```

2.3 Manipulação de Dados e Automação com Python

A linguagem Python tem se destacado na área de automação devido à sua facilidade de uso e à vasta quantidade de bibliotecas disponíveis para manipulação de dados. No contexto do RH, bibliotecas como Pandas, Psycopg2 e OpenPyXL desempenham um papel essencial na extração, transformação e geração de relatórios automatizados.

A biblioteca Pandas permite manipular grandes volumes de dados de forma eficiente, facilitando operações como filtragem, agrupamento e análise estatística. Já o Psycopg2 é utilizado para conectar o Python ao PostgreSQL, possibilitando a extração de dados diretamente do banco de dados.

Além disso, o OpenPyXL possibilita a criação e edição de arquivos Excel, permitindo que relatórios sejam gerados automaticamente e formatados conforme as necessidades da empresa. O exemplo abaixo ilustra a extração de dados do banco e a geração de um relatório Excel:

```
import psycopg2
import pandas as pd
from openpyxl import Workbook

Conexão com o banco de dados

conexao = psycopg2.connect("dbname=empresa user=admin password=1234")
cursor = conexao.cursor()

Consulta SQL

query = "SELECT nome, cargo, salario FROM funcionarios WHERE status = 'Ativo'"
cursor.execute(query)
```

```
dados = cursor.fetchall()

Criando DataFrame

colunas = ['Nome', 'Cargo', 'Salario']
df = pd.DataFrame(dados, columns=colunas)

Gerando arquivo Excel

wb = Workbook()
ws = wb.active
for r in dataframe_to_rows(df, index=False, header=True):
ws.append(r)
wb.save("relatorio_rh.xlsx")

Fechando conexão

cursor.close()
conexao.close()
```

Por fim, a automação do envio de relatórios por e-mail pode ser realizada com a biblioteca SMTPLib, garantindo que os relatórios sejam entregues automaticamente aos responsáveis, sem necessidade de intervenção manual.

Dessa forma, a integração de Banco de Dados, Python e automação possibilita um ganho significativo na eficiência dos processos do RH, permitindo que as empresas tomem decisões mais embasadas e estratégicas.

3 Estrutura da Base de Dados

3.1 Estrutura da Base de Dados

A organização e estruturação da base de dados desempenham um papel fundamental na gestão eficiente das informações de Recursos Humanos. O projeto foi desenvolvido com um banco de dados relacional, garantindo a integridade e a consistência das informações armazenadas. Para isso, optou-se pelo uso de um Sistema de Gerenciamento de Banco de Dados (SGBD), como MySQL ou PostgreSQL, que oferecem confiabilidade e escalabilidade.

A estrutura da base de dados foi modelada para contemplar as principais necessidades da gestão de pessoal, incluindo o armazenamento de dados dos colaboradores, cargos, departamentos, contratos, folha de pagamento e histórico de mudanças funcionais. Cada entidade foi representada por uma tabela, com chaves primárias e estrangeiras assegurando a correta interligação entre os dados.

```sql
-- Criando a tabela Departamentos primeiro
CREATE TABLE Departamentos (
    id_departamento INT PRIMARY KEY AUTO_INCREMENT,
    nome_departamento VARCHAR(100) NOT NULL,
    id_gerente INT,
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Criando a tabela de Cargos
CREATE TABLE Cargos (
    id_cargo INT PRIMARY KEY AUTO_INCREMENT,
    nome_cargo VARCHAR(100) NOT NULL,
    salario_base DECIMAL(10,2),
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Criando a tabela de Contrato
CREATE TABLE Contrato (
    id_contrato INT NOT NULL,
    ano_mes_referencia DATE NOT NULL,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) NOT NULL,  -- CPF sem formatação
    sexo CHAR(1) NOT NULL,
    data_nascimento DATE NOT NULL,
    endereco TEXT,
    telefone VARCHAR(15),
    email VARCHAR(100),
    data_admissao DATE NOT NULL,
    id_cargo INT,
    id_departamento INT,
    situacao_funcional VARCHAR(50),
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_contrato, ano_mes_referencia),
    FOREIGN KEY (id_cargo) REFERENCES Cargos(id_cargo),
    FOREIGN KEY (id_departamento) REFERENCES Departamentos(id_departamento)
);
```

Figura 1 – Criação das Tabelas - 1

```sql
-- Criando a tabela de Histórico de Cargos
CREATE TABLE Historico_Cargos (
    id_historico INT PRIMARY KEY AUTO_INCREMENT,
    id_contrato INT NOT NULL,
    ano_mes_referencia DATE NOT NULL,
    id_cargo_anterior INT,
    id_cargo_novo INT NOT NULL,
    data_alteracao DATE NOT NULL,
    motivo TEXT,
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_contrato, ano_mes_referencia) REFERENCES Contrato(id_contrato, ano_mes_referencia) ON DELETE CASCADE,
    FOREIGN KEY (id_cargo_anterior) REFERENCES Cargos(id_cargo),
    FOREIGN KEY (id_cargo_novo) REFERENCES Cargos(id_cargo)
);

-- Criando a tabela de Histórico de Situação Funcional
CREATE TABLE Historico_Situacao_Funcional (
    id_historico INT PRIMARY KEY AUTO_INCREMENT,
    id_contrato INT NOT NULL,
    ano_mes_referencia DATE NOT NULL,
    situacao_anterior VARCHAR(50),
    situacao_nova VARCHAR(50) NOT NULL,
    data_alteracao DATE NOT NULL,
    motivo TEXT,
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_contrato, ano_mes_referencia) REFERENCES Contrato(id_contrato, ano_mes_referencia) ON DELETE CASCADE
);

-- Criando a tabela de Folha de Pagamento
CREATE TABLE Folha_Pagamento (
    id_folha INT PRIMARY KEY AUTO_INCREMENT,
    id_contrato INT NOT NULL,
    ano_mes_referencia DATE NOT NULL,
    salario_bruto DECIMAL(10,2) NOT NULL,
    descontos DECIMAL(10,2),
    salario_liquido DECIMAL(10,2),
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_contrato, ano_mes_referencia) REFERENCES Contrato(id_contrato, ano_mes_referencia) ON DELETE CASCADE
);
```

Figura 2 – Criação das Tabelas - 2

```sql
-- Criando a tabela de Férias
CREATE TABLE Ferias (
    id_ferias INT PRIMARY KEY AUTO_INCREMENT,
    id_contrato INT NOT NULL,
    ano_mes_referencia DATE NOT NULL,
    inicio_periodo DATE NOT NULL,
    fim_periodo DATE NOT NULL,
    dias_ferias INT NOT NULL,
    aprovado BOOLEAN DEFAULT FALSE,
    user_modificacao VARCHAR(50),
    dt_ult_modificacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_contrato, ano_mes_referencia) REFERENCES Contrato(id_contrato, ano_mes_referencia) ON DELETE CASCADE
);
```

Figura 3 – Criação das Tabelas - 3

A tabela de departamentos armazena os setores da empresa, incluindo um campo para identificação do gerente responsável. A tabela de cargos registra as diferentes funções desempenhadas pelos colaboradores, associando um salário base a cada uma delas. Já a tabela de contratos contém informações detalhadas sobre cada colaborador, como dados pessoais, cargo ocupado, departamento de alocação e status funcional.

Além disso, foram criadas tabelas auxiliares para registrar históricos de mudanças na organização. A tabela de histórico de cargos documenta alterações de função dentro da empresa, registrando a data da mudança e o motivo. Da mesma forma, a tabela de histórico de situação funcional armazena modificações no status do colaborador, como promoções, afastamentos ou desligamentos.

A folha de pagamento foi estruturada para armazenar os valores salariais, incluindo remuneração bruta, descontos e salário líquido. Para o controle de férias, foi implementada uma tabela específica contendo períodos de descanso concedidos, duração e status de aprovação.

A modelagem dessas tabelas e seus relacionamentos permite uma gestão eficiente dos dados, garantindo a rastreabilidade das informações e facilitando a extração de relatórios gerenciais. O modelo entidade-relacionamento adotado ilustra a interconexão entre as tabelas, demonstrando como os dados são organizados para suportar as operações da empresa de forma estruturada e eficiente.

A implementação da base de dados foi realizada com SQL, utilizando o Neon Console para a criação da base de dados online. A criação automatizada das tabelas e a inserção de dados foram feitas de forma programática. Além disso, a integração com a aplicação foi feita utilizando a biblioteca `SQLAlchemy` no Python, possibilitando consultas dinâmicas e manipulação dos registros de maneira eficiente.

A correta estruturação da base de dados garante a integridade das informações e viabiliza a automação de processos como cálculos de folha de pagamento, controle de férias e geração de relatórios. A próxima seção abordará a implementação desses processos, destacando a automação desenvolvida com Python.

4 Desenvolvimento da Solução

4.1 Desenvolvimento da Solução

O desenvolvimento da solução proposta para a automação do gerenciamento de Recursos Humanos foi conduzido com o objetivo de otimizar a coleta e processamento de dados, além de proporcionar uma forma eficiente de gerar relatórios e realizar o envio automatizado de e-mails. A arquitetura da solução é baseada na utilização de Python, uma linguagem de programação versátil, juntamente com bibliotecas especializadas, que garantem a eficácia e a escalabilidade do sistema. A seguir, são detalhadas as etapas fundamentais do processo de desenvolvimento.

4.1.1 Conexão com o Banco de Dados

A primeira etapa crucial do desenvolvimento envolveu a conexão ao banco de dados, que foi configurado no Neon Console, uma plataforma robusta para a gestão de bases de dados PostgreSQL. Utilizou-se a biblioteca `psycopg2` para realizar a integração entre o código Python e o banco de dados online. A conexão foi estabelecida de forma segura, configurando adequadamente os parâmetros como host, porta, nome do banco, usuário e senha, garantindo assim o acesso aos dados armazenados na plataforma.

A arquitetura do banco de dados, projetada com base em um modelo relacional, contém informações fundamentais sobre os colaboradores, cargos, departamentos, situações funcionais e históricos de mudanças. Essa estrutura foi cuidadosamente pensada para atender às necessidades de gestão e garantir a integridade dos dados durante o processo de consulta e manipulação.

4.1.2 Consultas SQL para Extração de Dados

Com a conexão estabelecida, o próximo passo foi a formulação de consultas SQL para extrair dados relevantes do banco de dados, que serviriam como base para os relatórios gerenciais. As consultas foram elaboradas para abordar aspectos cruciais da gestão de recursos humanos, como o cálculo da taxa de turnover mensal, a distribuição de funcionários por departamento e o perfil demográfico dos colaboradores, incluindo idade e gênero.

A consulta para o turnover mensal, por exemplo, calcula o número de demissões em relação ao total de funcionários ativos, gerando uma taxa percentual que pode ser usada para análise de rotatividade na empresa. A consulta sobre a distribuição de funcionários por departamento organiza as informações por setores, permitindo um melhor entendimento

da alocação de pessoal. Já a consulta sobre o perfil demográfico analisa dados como gênero e idade média, fornecendo insights importantes sobre a composição da força de trabalho.

Essas consultas foram parametrizadas para utilizar o ano e mês atual, garantindo que os relatórios gerados fossem sempre atualizados conforme o período em questão. A utilização de parâmetros dinâmicos nas consultas SQL torna o sistema flexível e adaptável a diferentes períodos de tempo, o que facilita a continuidade das operações ao longo do tempo.

4.1.3 Geração de Relatórios com Pandas e Exportação para Excel

Após a extração dos dados, foi utilizada a biblioteca `pandas` para o processamento dos resultados das consultas. O `pandas` é uma biblioteca amplamente utilizada para análise de dados em Python, fornecendo estruturas de dados e funções eficientes para manipulação, filtragem e agregação de dados. Com o auxílio do `pandas`, foi possível organizar os resultados das consultas SQL em DataFrames, que são estruturas tabulares adequadas para análise e processamento de dados.

Uma vez organizados os dados, os relatórios foram gerados em formato Excel, utilizando a biblioteca `openpyxl`. Cada consulta foi armazenada em uma aba distinta dentro do arquivo Excel, facilitando a visualização e interpretação das informações. O uso do formato Excel não apenas permite uma visualização estruturada dos dados, mas também oferece a flexibilidade de posterior análise em ferramentas como Microsoft Excel ou Google Sheets. Essa abordagem de geração de relatórios torna o processo mais acessível e prático para os gestores de RH, que podem utilizar os relatórios para a tomada de decisões estratégicas.

4.1.4 Implementação do Envio Automatizado de E-mails

A última etapa do desenvolvimento envolveu a automação do envio de e-mails, permitindo que os relatórios gerados fossem enviados automaticamente para os responsáveis pelo acompanhamento dos dados. Para isso, foi implementada uma função que utiliza o protocolo SMTP para o envio de e-mails, com a biblioteca `smtplib`. A solução foi configurada para enviar os relatórios gerados como anexos, diretamente para o e-mail do responsável pela área de Recursos Humanos.

O envio de e-mails foi automatizado para garantir que os relatórios chegassem de forma eficiente e pontual aos destinatários. Para isso, o código foi estruturado de forma a permitir a personalização do corpo do e-mail e a inclusão de anexos, como o arquivo Excel contendo os relatórios. A segurança da comunicação foi garantida através da utilização de criptografia TLS durante o processo de envio do e-mail.

A automatização desse processo não apenas economiza tempo, mas também minimiza a possibilidade de erros humanos, assegurando que os relatórios sejam enviados de forma consistente e em tempo hábil. Essa funcionalidade representa uma significativa melhoria na eficiência operacional, permitindo que a equipe de RH tenha acesso imediato às informações necessárias para a gestão de pessoal.

4.1.5 Considerações Finais

O desenvolvimento dessa solução envolveu a integração de diversas tecnologias e bibliotecas de Python, com ênfase na automação de processos cruciais para a gestão de Recursos Humanos. A utilização do Neon Console para a criação da base de dados online, associada ao uso das bibliotecas `psycopg2`, `pandas`, `openpyxl` e `smtplib`, permitiu a construção de um sistema robusto e eficiente para a extração, processamento e envio de relatórios gerenciais.

A automatização do processo de geração de relatórios e envio de e-mails contribui para a otimização do tempo da equipe de RH, permitindo que se concentrem em tarefas mais estratégicas e de maior valor agregado. A solução desenvolvida é altamente escalável, podendo ser adaptada para diferentes necessidades da organização, garantindo flexibilidade e eficiência na gestão de dados.

A seguir, é apresentado o código utilizado para a implementação da solução, de forma ilustrada nas Figuras 4, 5, 6, 7, 8 e 9.

```
!pip install psycopg2
!pip install psycopg2 pandas openpyxl

import pandas as pd
import psycopg2
from datetime import datetime
```

```
[67]
host = "ep-blue-rain-a5lv8u2n-pooler.us-east-2.aws.neon.tech"
port = "5432"  # geralmente é 5432
database = "neondb"
user = "neondb_owner"
password = "npg_VMOcWfYoE45U"
```

Figura 4 – Código Python - 1

```python
[68] # Função para executar a consulta SQL e retornar os resultados como DataFrame
     def execute_query(query):
         try:
             connection = psycopg2.connect(
                 host=host,
                 port=port,
                 dbname=database,
                 user=user,
                 password=password
             )
             cursor = connection.cursor()
             cursor.execute(query)
             # Recuperar os resultados e colocar em um DataFrame
             columns = [desc[0] for desc in cursor.description]
             rows = cursor.fetchall()
             df = pd.DataFrame(rows, columns=columns)
             cursor.close()
             connection.close()
             return df
         except Exception as e:
             print("Erro ao executar consulta:", e)
             return None

     # Obtendo ano e mês atual
     current_year_month = datetime.now().strftime("%Y-%m")  # Ex: '2025-01'
     first_day_of_month = datetime.now().replace(day=1).strftime("%Y-%m-%d")  # Ex: '2025-01-01'
```

Figura 5 – Código Python - 2

```python
# Consultas SQL com base no ano e mês atual
queries = {
    "turn_over_mensal": f"""
        SELECT
            COUNT(CASE WHEN hs.situacao_nova = 'Demitido' THEN 1 END) AS total_demitidos,
            COUNT(DISTINCT c.id_contrato) AS total_funcionarios,
            (COUNT(CASE WHEN hs.situacao_nova = 'Demitido' THEN 1 END) * 100.0) /
            NULLIF(COUNT(DISTINCT c.id_contrato), 0) AS taxa_turnover_percent
        FROM
            Contrato c
        LEFT JOIN
            Historico_Situacao_Funcional hs
            ON c.id_contrato = hs.id_contrato
            AND hs.ano_mes_referencia = '{first_day_of_month}'  -- Usando o primeiro dia do mês corrente
        WHERE
            c.ano_mes_referencia = '{first_day_of_month}';  -- Usando o primeiro dia do mês corrente
    """,
    "distribuicao_func_por_departamento": """
        SELECT
            d.nome_departamento,
            COUNT(c.id_contrato) AS total_funcionarios
        FROM
            Departamentos d
        LEFT JOIN
            Contrato c ON d.id_departamento = c.id_departamento
        WHERE
            c.situacao_funcional = 'Ativo'
        GROUP BY
            d.nome_departamento
        ORDER BY
            total_funcionarios DESC;
    """,
    "perfil_demografico": """
        SELECT
            sexo,
            COUNT(id_contrato) AS total,
            ROUND(AVG(EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM data_nascimento)), 1) AS idade_media
        FROM
            Contrato
        WHERE
            situacao_funcional = 'Ativo'
        GROUP BY sexo;
    """
}
```

Figura 6 – Código Python - 3

```
[68] # Executando todas as consultas e criando um DataFrame para cada relatório
     dfs = {}
     for report_name, query in queries.items():
         print(f"Executando consulta para {report_name}...")
         df = execute_query(query)
         if df is not None:
             dfs[report_name] = df

     # Salvando os resultados em um arquivo Excel com múltiplas abas
     if dfs:
         with pd.ExcelWriter("relatorios_empresa.xlsx", engine="openpyxl") as writer:
             for report_name, df in dfs.items():
                 # Renomeando as abas para garantir que o título tenha 31 caracteres ou menos
                 safe_report_name = report_name[:31]  # Limita o nome da aba a 31 caracteres
                 df.to_excel(writer, sheet_name=safe_report_name, index=False)
         print("Relatórios exportados para 'relatorios_empresa.xlsx'")
     else:
         print("Não foi possível gerar os relatórios.")
```

Figura 7 – Código Python - 4

```python
[73] import smtplib
     from email.mime.multipart import MIMEMultipart
     from email.mime.text import MIMEText
     from email.mime.base import MIMEBase
     from email import encoders

     # Função para enviar e-mail com anexo
     def send_email(subject, body, to_email, attachment_path):
         from_email = "empresa.teste.automatizacao@gmail.com"  # Seu e-mail
         password = "mcmt zqrq lfcw zlev"  # Senha de aplicativo gerada

         # Criando a mensagem MIME
         msg = MIMEMultipart()
         msg['From'] = from_email
         msg['To'] = to_email
         msg['Subject'] = subject

         # Corpo do e-mail
         msg.attach(MIMEText(body, 'plain'))

         # Anexando o arquivo
         part = MIMEBase('application', "octet-stream")
         with open(attachment_path, "rb") as f:
             part.set_payload(f.read())
         encoders.encode_base64(part)
         part.add_header('Content-Disposition', 'attachment; filename="relatorios_empresa_atualizado.xlsx"')
         msg.attach(part)

         # Enviando o e-mail via SMTP
         try:
             # Conexão com o servidor SMTP do Gmail
             server = smtplib.SMTP('smtp.gmail.com', 587)
             server.starttls()  # Criptografia
             server.login(from_email, password)
             server.sendmail(from_email, to_email, msg.as_string())
             server.quit()
             print("E-mail enviado com sucesso!")
         except Exception as e:
             print(f"Erro ao enviar e-mail: {e}")

     # Corpo do e-mail
     email_body = "Prezado(a),\n\nSegue o relatório de RH gerado para o mês atual.\n\nAtenciosamente,\nEquipe RH"
```

Figura 8 – Código Python - 5

```python
# Destinatário (e-mail do RH)
rh_email = "gabrielhsmaia@gmail.com"  # Substitua pelo e-mail do RH

# Caminho do arquivo a ser enviado
file_path = "relatorios_empresa.xlsx"

# Enviar o e-mail com o relatório
send_email("Relatório RH - Mês Atual", email_body, rh_email, file_path)
```

Figura 9 – Código Python - 6

5 Resultados e Testes

5.1 Resultados e Testes

Nesta seção, são apresentados os resultados obtidos com a implementação da solução desenvolvida, bem como os testes realizados para validar a extração dos dados, a geração dos relatórios e o envio automatizado de e-mails. Além disso, discute-se os benefícios proporcionados pela automação para a empresa.

5.1.1 Exemplos de Relatórios Gerados

A extração e o processamento dos dados resultaram na geração de relatórios gerenciais em formato Excel, permitindo uma análise clara e estruturada das informações relacionadas à gestão de Recursos Humanos. Os relatórios incluem indicadores essenciais, tais como:

- **Taxa de Turnover Mensal:** Apresenta a relação entre o número de demissões e o total de funcionários ativos no período analisado.
- **Distribuição de Funcionários por Departamento:** Exibe a quantidade de colaboradores alocados em cada setor da empresa, auxiliando na análise da estrutura organizacional.
- **Perfil Demográfico dos Colaboradores:** Inclui dados sobre a distribuição de funcionários por gênero e a média de idade, fornecendo insights sobre a composição da força de trabalho.

As Figuras 10, 11 e 12 ilustram alguns dos relatórios gerados, demonstrando a eficácia da abordagem adotada.

A	B	C
total_demitidos	total_funcionarios	taxa_turnover_percent
6	50	12.000000000000000

Figura 10 – Relatório turnover

A	B
nome_departamento	**total_funcionarios**
Vendas	3
Compras	3
Financeiro	3
Jurídico	3
Logística	3
Marketing	3
Operações	3
RH	3
TI	3
Auditoria	3
Serviços Gerais	2
Pesquisa e Desenvolvimento	2
Comunicação	2
Controle de Qualidade	2
Estratégia	2
Planejamento	2
Inovação	2
Produção	2
Treinamento	2
Serviço ao Cliente	2

Figura 11 – Relatório departamentos

A	B	C
sexo	total	idade_media
F	25	38.4
M	25	33.0

Figura 12 – Relatório demográfico

5.1.2 Validação dos Dados Extraídos

Para garantir a integridade e a confiabilidade dos dados apresentados nos relatórios, foi realizada uma validação criteriosa dos resultados obtidos. Essa validação incluiu:

- Comparação dos valores extraídos com os registros armazenados no banco de dados para verificar a consistência dos dados.
- Testes manuais utilizando consultas SQL independentes para confrontar os resultados obtidos automaticamente pelo sistema.
- Análise da estrutura das tabelas e das chaves primárias para evitar duplicidades ou informações incorretas nos relatórios.

Os testes de validação confirmaram que os dados extraídos refletem corretamente as informações armazenadas no banco, garantindo a precisão dos relatórios gerados.

5.1.3 Testes do Envio Automático de E-mails

A funcionalidade de envio automatizado de e-mails foi testada para assegurar que os relatórios fossem entregues corretamente aos destinatários. Os testes incluíram:

- Verificação da autenticidade da conexão SMTP, garantindo que os e-mails fossem enviados de forma segura.
- Testes com diferentes provedores de e-mail para confirmar a compatibilidade e a entrega correta das mensagens.
- Análise do tempo de processamento e envio, verificando a eficiência da solução implementada.

A Figura 13 apresenta um exemplo de e-mail recebido com o relatório anexado, evidenciando o funcionamento adequado do sistema.

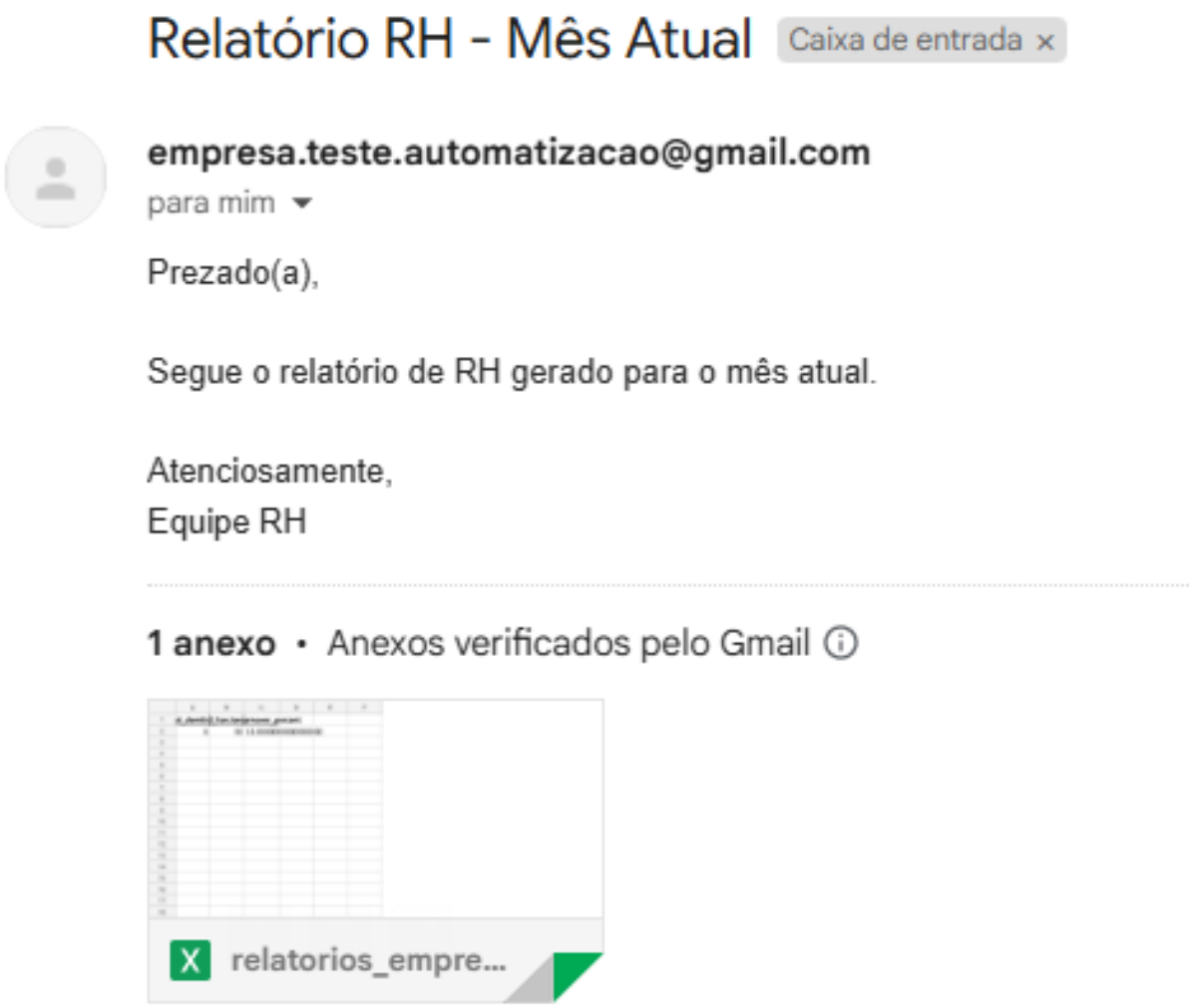

Figura 13 – E-mail enviado

5.1.4 Benefícios da Automação para a Empresa

A implementação da solução automatizada trouxe diversos benefícios para a empresa, destacando-se:

- **Redução do Tempo de Processamento:** A extração de dados e a geração de relatórios, que anteriormente exigiam tempo e esforço manual, agora ocorrem de forma automatizada e rápida.
- **Minimização de Erros Humanos:** A automação elimina falhas decorrentes da manipulação manual dos dados, garantindo maior precisão nos relatórios.
- **Entrega Pontual de Informações:** Os relatórios são enviados automaticamente aos responsáveis em tempo hábil, facilitando a tomada de decisões estratégicas.
- **Escalabilidade da Solução:** A metodologia adotada pode ser facilmente expandida para abranger novos indicadores e relatórios conforme necessário.

Dessa forma, a automação do processo de geração e envio de relatórios representa um avanço significativo para a gestão de Recursos Humanos, promovendo maior eficiência e confiabilidade nas análises realizadas.

6 Conclusão

6.1 Conclusão

Este trabalho apresentou o desenvolvimento e a implementação de um sistema automatizado para a extração de dados, geração de relatórios gerenciais e envio automático de e-mails, com foco na otimização do processo de gestão de Recursos Humanos. A solução desenvolvida demonstrou ser eficiente na redução do tempo necessário para a obtenção de informações estratégicas, garantindo maior precisão e confiabilidade dos dados.

6.1.1 Resumo dos Resultados Alcançados

Os testes realizados confirmaram o correto funcionamento da solução, validando a precisão dos dados extraídos e a consistência dos relatórios gerados. Além disso, o envio automatizado de e-mails foi testado com sucesso, assegurando que os relatórios fossem entregues aos destinatários de forma eficiente e segura. Entre os principais resultados alcançados, destacam-se:

- Redução significativa do tempo de extração e processamento de dados.
- Geração de relatórios precisos e estruturados em formato Excel.
- Eliminação de erros manuais na consolidação das informações.
- Automação do envio de relatórios, garantindo entrega pontual e segura.

Esses resultados demonstram a viabilidade da solução proposta e sua relevância para a otimização do fluxo de trabalho na empresa.

6.1.2 Impacto da Automação no Processo de Envio de Relatórios

A automação do processo de extração, geração e envio de relatórios trouxe benefícios diretos para a equipe responsável pela gestão de Recursos Humanos. Antes da implementação do sistema, a coleta e análise de dados exigiam um esforço manual considerável, sujeito a falhas e atrasos. Com a automatização, foi possível garantir:

- Maior agilidade na tomada de decisões, pois os relatórios são gerados e enviados automaticamente.
- Redução da carga operacional dos funcionários, permitindo que se concentrem em atividades mais estratégicas.

- Maior confiabilidade e segurança dos dados, evitando inconsistências e atrasos na comunicação das informações.

Dessa forma, a implementação do sistema não apenas otimizou o tempo da equipe, mas também melhorou a qualidade e a acessibilidade dos dados para a gestão organizacional.

6.1.3 Possíveis Melhorias e Extensões Futuras

Embora a solução implementada tenha se mostrado eficaz, algumas melhorias e extensões podem ser exploradas para aumentar ainda mais sua eficiência e aplicabilidade. Entre as possíveis evoluções, destacam-se:

- **Integração com outras plataformas:** A implementação de APIs para integração com sistemas de gestão (ERP) poderia ampliar a interoperabilidade da solução.
- **Geração de relatórios interativos:** A adoção de ferramentas como `Dash` ou `Power BI` permitiria uma visualização mais dinâmica e interativa dos dados.
- **Monitoramento de envio de e-mails:** A inclusão de um sistema de logs detalhados possibilitaria o acompanhamento em tempo real da entrega dos relatórios.
- **Automação avançada baseada em inteligência artificial:** Modelos de aprendizado de máquina poderiam ser incorporados para prever padrões e sugerir ações estratégicas com base nos dados históricos.

Essas melhorias podem ampliar a aplicabilidade da solução e garantir ainda mais eficiência para a gestão de Recursos Humanos, tornando o processo de análise de dados mais ágil, seguro e estratégico.

Conclui-se, portanto, que a automação do processo de geração e envio de relatórios representa um avanço significativo na otimização do gerenciamento de dados, proporcionando maior eficiência, redução de erros e melhor suporte à tomada de decisões. A metodologia utilizada permitiu a construção de um sistema flexível e escalável, que pode ser expandido conforme as necessidades futuras da organização.

7 Referências

Severance, Charles R. Python para Todos: Explorando Dados com Python 3. CreateSpace Independent Publishing Platform, 2016.

McLaughlin, Brett; Pollice, Gary; West, David. Head First Object-Oriented Analysis and Design: A Brain Friendly Guide to OOAD. O'Reilly Media, 2006.

Beaulieu, Alan. Use a Cabeça! SQL. Alta Books, 2007.